

Docker Networking Lab

How to Run the Three-Tier Application (Frontend → Backend → Database)

Overview

This project is a three-tier Docker application used to practice container networking. It consists of an Nginx frontend, a Node.js/Express backend, and a MySQL database. The steps below walk through building the images, running them on a bridge network, comparing host networking, and deploying them on a Docker Swarm overlay network.

Tier	Technology	Port	Purpose
Frontend	Nginx	80	Serves index.html, proxies /api/ to backend
Backend	Node.js / Express	5000	Exposes /api/health and /api/students, queries MySQL
Database	MySQL 8.0	3306	Stores the students table (seeded by init.sql)

Prerequisites: Docker Desktop (or Docker Engine) installed and running, with Docker Compose available. Step 1 below covers downloading the lab folder.

This guide covers Windows, Mac, and Linux. Commands are identical across all three except for multi-line commands, where each terminal uses a different line-continuation character, and the file path used to mount **init.sql** into the database container.

Line-continuation characters by terminal

Terminal	Continuation character	Current-folder variable
Mac / Linux Terminal (bash/zsh)	\ (backslash)	\$(pwd)
Windows PowerShell	` (backtick)	\${PWD}
Windows Command Prompt (cmd.exe)	^ (caret)	%cd%

Tip: to avoid this entirely, any multi-line command can be typed as a single line instead — see the one-line versions given alongside Step 4 below.

Step 1 — Download the Lab Folder

Download the lab project (frontend, backend, and database folders) from GitHub:

```
https://github.com/Riyabhimani/Teaching_Assistant_ComputerNetworks/blob/main/Lab15_Docker/Lab15_Docker.zip
```

Note: the archive extracts into a **nested** folder — unzipping **Lab15_Docker.zip** produces **lab_docker/lab_docker/**, and it's the inner **lab_docker** folder that actually contains **frontend/**, **backend/**, and **database/**. Change into that inner folder before running any commands.

Mac / Linux (Terminal):

```
unzip Lab15_Docker.zip
cd lab_docker/lab_docker
```

Windows (PowerShell):

```
Expand-Archive Lab15_Docker.zip -DestinationPath .
cd lab_docker\lab_docker
```

Or simply click the "Code" button on the GitHub page (or open the .zip link above), choose "Download ZIP", then right-click the downloaded file and choose "Extract All" (Windows) or double-click it (Mac) to unzip. Then open the inner **lab_docker** folder and confirm it contains **frontend/**, **backend/**, and **database/** subfolders before continuing.

Step 2 — Build the Frontend and Backend Images

The database uses the official MySQL image directly, so only the frontend and backend need to be built. Run these from the inner **lab_docker** folder in a terminal (PowerShell or Command Prompt on Windows; Terminal on Mac/Linux).

```
cd lab_docker/backend
docker build -t lab-backend:1.0 .

cd ../frontend
docker build -t lab-frontend:1.0 .
```

Verify both images were created:

```
docker images | grep lab-
```

Step 3 — Create and Inspect a Bridge Network

```
docker network create lab-bridge-net
docker network inspect lab-bridge-net
```

The inspect output should show the subnet/gateway and, once containers are attached, each container's name and IP address under the network's "Containers" section.

Step 4 — Launch the Three Containers on the Bridge Network

Start the database first, since the backend depends on it. The only line that differs by OS is the **-v** mount path for **init.sql** — use the version that matches your system.

Mac / Linux (Terminal) — multi-line:

```
docker run -d --name database --network lab-bridge-net \
  -e MYSQL_ROOT_PASSWORD=rootpass \
  -e MYSQL_DATABASE=labdb \
  -e MYSQL_USER=labuser \
  -e MYSQL_PASSWORD=labpassword \
  -v "$(pwd)/database/init.sql:/docker-entrypoint-initdb.d/init.sql" \
  mysql:8.0
```

Mac / Linux (Terminal) — single-line:

```
docker run -d --name database --network lab-bridge-net -e MYSQL_ROOT_PASSWORD=rootpass -e
MYSQL_DATABASE=labdb -e MYSQL_USER=labuser -e MYSQL_PASSWORD=labpassword -v "${PWD}/database/
init.sql:/docker-entrypoint-initdb.d/init.sql" mysql:8.0
```

Windows PowerShell — multi-line:

```
docker run -d --name database --network lab-bridge-net `
-e MYSQL_ROOT_PASSWORD=rootpass `
-e MYSQL_DATABASE=labdb `
-e MYSQL_USER=labuser `
-e MYSQL_PASSWORD=labpassword `
-v "${PWD}/database/init.sql:/docker-entrypoint-initdb.d/init.sql" `
mysql:8.0
```

Windows PowerShell — single-line:

```
docker run -d --name database --network lab-bridge-net -e MYSQL_ROOT_PASSWORD=rootpass -e
MYSQL_DATABASE=labdb -e MYSQL_USER=labuser -e MYSQL_PASSWORD=labpassword -v "${PWD}/database/
init.sql:/docker-entrypoint-initdb.d/init.sql" mysql:8.0
```

Windows Command Prompt (cmd.exe) — multi-line:

```
docker run -d --name database --network lab-bridge-net ^
-e MYSQL_ROOT_PASSWORD=rootpass ^
-e MYSQL_DATABASE=labdb ^
-e MYSQL_USER=labuser ^
-e MYSQL_PASSWORD=labpassword ^
-v "%cd%/database/init.sql:/docker-entrypoint-initdb.d/init.sql" ^
mysql:8.0
```

Windows Command Prompt (cmd.exe) — single-line:

```
docker run -d --name database --network lab-bridge-net -e MYSQL_ROOT_PASSWORD=rootpass -e
MYSQL_DATABASE=labdb -e MYSQL_USER=labuser -e MYSQL_PASSWORD=labpassword -v "%cd%/database/
init.sql:/docker-entrypoint-initdb.d/init.sql" mysql:8.0
```

Wait about 15–20 seconds for MySQL to initialize, then start the backend and frontend (identical on every OS):

```
docker run -d --name backend --network lab-bridge-net lab-backend:1.0
```

```
docker run -d --name frontend --network lab-bridge-net -p 8080:80 lab-frontend:1.0
```

Open <http://localhost:8080> in a browser. The page calls the backend's **/api/health** and **/api/students** endpoints, and the backend reaches MySQL by container name (**database**) using Docker's embedded DNS — this confirms bridge-network name resolution is working.

Step 5 — Verify Communication Between Containers

```
# Confirm all three containers are attached to the network
docker network inspect lab-bridge-net
```

```
# From inside the frontend, resolve and reach the backend by name
docker exec -it frontend ping -c 2 backend
docker exec -it frontend wget -qO- http://backend:5000/api/health
```

```
# From inside the backend, resolve and reach the database by name
```

```
docker exec -it backend ping -c 2 database
```

Step 6 — Configure Host Networking and Compare

Stop the bridge-network frontend first to free up port 80, then run a container using the host network driver:

```
docker stop frontend
docker run -d --name frontend-host --network host lab-frontend:1.0
```

Visit **<http://localhost:80>** directly — no -p mapping is needed or possible, because the container shares the host's network namespace.

Key differences to note for the lab writeup:

- Bridge: container gets its own private IP; requires -p to expose ports to the host; container names resolve via Docker's embedded DNS.
- Host: container shares the host's network stack directly; no port mapping or NAT; no isolated container IP; -p flags are ignored; container-name DNS resolution behaves differently since there is no separate bridge network to resolve names on.

Clean up before the next step:

```
docker rm -f frontend-host
```

Step 7 — Create an Overlay Network with Docker Swarm

```
# Initialize swarm mode (only needed once per host)
docker swarm init

# Create the overlay network referenced in swarm-stack.yml
docker network create -d overlay lab-overlay-net

# Deploy the stack
docker stack deploy -c swarm-stack.yml labstack
```

Check that the services came up:

```
docker service ls
docker service ps labstack_backend
docker service ps labstack_frontend
docker service ps labstack_database
```

Visit <http://localhost:8090> (the port mapped in `swarm-stack.yml`). The stack runs 2 replicas of the backend, showing that requests can be served by either replica while both still resolve **database** over the overlay network.

Tear down the stack when finished:

```
docker stack rm labstack
docker swarm leave --force # optional, only if done experimenting with swarm
```

Step 8 — Troubleshooting Commands

Use these commands to diagnose the connectivity issues called out in the lab objectives:

Issue	Command	What to look for
Incorrect network attachment	<code>docker network inspect <network></code>	Confirm the container appears under "Containers"; if missing, reconnect with <code>docker network connect <network> <container></code>
Missing port mapping	<code>docker port <container></code>	Should list host:container port bindings; if empty, the container was started without <code>-p</code> and won't be reachable from the host
DNS resolution between containers	<code>docker exec -it <c> nslookup <name></code> <code>docker exec -it <c> ping <name></code>	Name should resolve to the other container's overlay/bridge IP; failures usually mean the two containers are on different networks
Service accessibility	<code>docker logs <container></code> <code>docker inspect <container></code>	Check for connection-refused / timeout errors and confirm the container's networks and exposed ports

Note: the extracted zip contains a stray file named **database/init.sql**;C. It is not referenced by any Dockerfile or compose file and can be safely deleted — only **database/init.sql** is used to seed the database.